

SafeContract: Composable Safety Monitors for VLA Action Spaces

Krishnam Gupta
Independent Research
San Francisco, CA
kayjee1994@gmail.com

Abstract—Vision-Language-Action (VLA) models predict robot actions from images and instructions, but no composable, architecture-agnostic runtime safety framework exists for their outputs. We introduce SafeContract, a design-by-contract [5] framework that wraps any VLA policy with composable safety constraints via a Python decorator. We show that a lightweight safety contract not only prevents unsafe actions but reveals systematic violation patterns that characterize model behavior and detect distribution shift. Controlled ablations demonstrate zero false positives on clean policies while catching all violations on drifting and jerky policies. Across 130 LIBERO tasks (40 standard + 90 diverse), each task produces a distinct violation fingerprint across joint dimensions, with bounds violation rates varying from 4% to 37% and velocity violations dominating at 72.4% of all violations. Changes in violation rate detect task switches with $p < 0.005$ across all tested pairs. These findings suggest that action-space monitors double as interpretability tools: violation patterns expose what a VLA “thinks” about its action space without requiring model internals. The full decorator adds $\sim 13\mu\text{s}$ overhead per action, requiring no QP solver, VLM, or model retraining. We release the framework as an open-source toolkit.¹

Index Terms—VLA, safety monitoring, interpretability, edge deployment, design by contract

I. Introduction

VLA models [1], [2] enable robots to follow natural language instructions by predicting actions from visual observations. Deploying these models to real robots faces a critical safety gap: no runtime safety layer exists for VLA action outputs.

This is not theoretical. OpenVLA’s deployment script sends raw neural network outputs directly to motors with zero bounds checking [1]. pi0’s OpenPI streams actions via WebSocket with no validation [4]. LeRobot’s [3] EEBoundsAndSafety processor is the only existing runtime check, but it is imperative, provides no composition guarantees, and is easy to omit. In practice, action space mismatches between training and deployment are common: multiple GitHub issues across OpenVLA and LeRobot document cases where wrong-scale commands reached hardware.

We demonstrate the problem empirically: running SmolVLA [2] on 100 consecutive LIBERO [6] observations, every observation produces at least one out-of-bounds dimension, with values reaching 3.70 (Sec. IV-A).

Beyond enforcement, violation patterns carry information. A safety monitor that logs every violation creates a structured record of how a policy interacts with its constraint boundary. We show this record is surprisingly rich: different tasks produce distinct violation fingerprints across joints, and shifts in violation rate signal distribution change. This reframes safety contracts from passive enforcement to active monitoring tools.

We introduce SafeContract with three contributions:

- 1) A `@safety_contract` Python decorator that enforces composable safety constraints on any VLA `predict()` method at runtime (Properties 1–II-C).
- 2) Controlled ablations showing zero false positives on clean policies and full violation detection on corrupted ones (Sec. IV-B).
- 3) Violation fingerprinting across 130 LIBERO tasks showing task-specific joint-violation signatures and statistically significant shift detection (Sec. IV-C).

II. Method

A. Safety Contracts

A safety contract $C = (\mathbf{l}, \mathbf{u}, v_{\max})$ specifies per-joint action bounds and a velocity limit: $l_i \leq a_i \leq u_i$ for each joint i , and $\|\mathbf{a}_t - \mathbf{a}_{t-1}\|_{\infty} \leq v_{\max}$.

B. Enforcement Order

Enforcement proceeds in three phases: (1) clip to per-joint bounds, (2) clamp velocity (project each joint’s delta onto $[-v_{\max}, v_{\max}]$), (3) re-clip to bounds. The final re-clip is critical: velocity clamping shifts actions relative to the previous timestep, which can push them outside the original bounds. Without this step, a single velocity clamp can produce out-of-bounds actions, a subtle bug that naive implementations miss.

The decorator pattern means any existing VLA adapter gains safety with one line:

```
@safety_contract(action_range=[-1,1],
                  joint_velocity_max=0.1)
def predict(self, image, instruction):
    return self.model(image, instruction)
```

Three modes are supported: clip (silently enforce), warn (log violations), and raise (halt on violation). All violations are logged with timestep, dimension, magnitude, and severity.

¹Code: <https://github.com/krishnam94/vla-edge>

C. Composition Properties

Property 1 (Composition via Intersection). For hyperrectangular contracts C_1, C_2 , sequential clipping equals intersection clipping: $\text{clip}(\text{clip}(\mathbf{a}, C_1), C_2) = \text{clip}(\mathbf{a}, C_1 \cap C_2)$. Stacking multiple `@safety_contract` decorators is order-independent.

Proof. Follows from idempotence and commutativity of per-dimension min/max. Verified empirically: sequential clipping through two contracts matches intersection clipping on 200 random-walk sequences (1,482 violations). $\square$

Property 2 (Feasibility from Safe States). Let $\mathbf{a}_{t-1}$ satisfy contract C with $v_{\max} > 0$. Then the feasible set is non-empty, since the “hold position” action $\mathbf{a}_t = \mathbf{a}_{t-1}$ satisfies both bounds and velocity constraints.

Observation (Violation Rate Concentration). For a stationary policy π and fixed contract C , the per-step violation rate converges to a constant r^* . A sustained deviation $|\hat{r}_w - r^*| > \epsilon$ over window w signals non-stationarity in the policy’s output distribution. This is an empirical observation validated in Sec. IV-C.

III. Related Work

Training-time safety. SafeVLA [7] fine-tunes with constrained RL (83% violation cost reduction); RobustVLA [10] adds robust optimization for perturbation resilience. Both require retraining; SafeContract is orthogonal.

Inference-time enforcement. AEGIS [8] solves a CBF-QP at runtime (0.36 ms) using VLM scene understanding. SafeDec [9] applies STL-constrained decoding but requires logit access. Safety Chip [11] uses LTL monitors for plan-level constraints. SafeDiffuser [12] and CoDiG [13] embed barriers in diffusion denoising. ATACOM [14] projects onto constraint manifolds for Octo. Agent Behavioral Contracts [17] brings design-by-contract to AI agents but targets discrete LLM decisions, not continuous robot action spaces.

Runtime monitoring. Sentinel [18] combines temporal consistency with VLM monitoring to detect failures, but does not prevent unsafe actions and requires an additional VLM call. LIBERO-Plus [15] and VLA-Arena [16] benchmark VLA robustness under perturbation. SABER [19] demonstrates black-box adversarial attacks on VLAs; SafeContract provides defense-in-depth since enforcement operates at the action boundary regardless of input provenance.

SafeContract differs from all the above: (i) architecture-agnostic; (ii) explicit composition; (iii) pure clipping ($\sim 13 \mu\text{s}$); (iv) violation logging as behavioral telemetry.

IV. Experiments

Platform: Apple M3, CPU, Python 3.12. Full contract overhead: $15.6 \pm 3.8 \mu\text{s}$ (5,000 iterations), representing 0.000075% of SmolVLA inference time. Tighter contracts

SmolVLA outputs exceed safe bounds on LIBERO

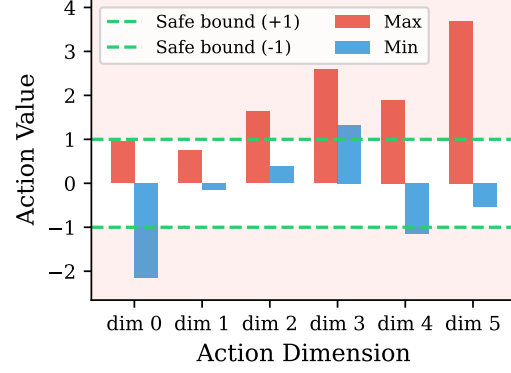


Fig. 1: SmolVLA action ranges per dimension on 100 LIBERO observations. All 6 dimensions exceed the $[-1, 1]$ bounds (green dashes), reflecting normalization mismatch between SO-100 training data and LIBERO’s action space.

TABLE I: Controlled ablation across policy types. Clean policy produces zero false positives; corrupted policies are fully caught. Naive np.clip catches bounds violations but misses all velocity violations.

Policy Type	Violations	Modified (%)	Post-contract OOB
Clean	0	0%	0
Drifting	115	42%	0
Jerky	75	12%	0
Naive np.clip	0 caught	–	199 vel. missed

catch more violations with proportionally more clipping; at the moderate operating point (± 1.0 bounds, $v_{\max} = 0.1$), mean correction is 0.45 units per action.

A. VLAs Produce Unsafe Actions (EXP-A)

We run SmolVLA on 100 consecutive LIBERO observations. Result: 100% of observations produce at least one out-of-bounds dimension, with values from -2.16 to 3.70 (expected $[-1, 1]$). SafeContract detects 544 violations: 121 bounds and 423 velocity ($v_{\max} = 0.1$).

Normalization caveat. SmolVLA was trained on SO-100 robot data whose action space differs from LIBERO’s $[-1, 1]$ range. The 100% OOB rate primarily reflects this normalization mismatch, precisely the class of invisible deployment error SafeContract catches.

B. Controlled Ablation (EXP-G)

To establish monitor precision, we test three synthetic policy types on 100-step sequences (7-DOF, bounds $[-1, 1]$, $v_{\max} = 0.1$):

The clean policy generates actions within bounds with smooth velocity. SafeContract produces zero violations and zero modifications: a safe policy passes through untouched. The drifting policy (cumulative Gaussian noise, simulating distribution shift) produces 115 violations; 42%

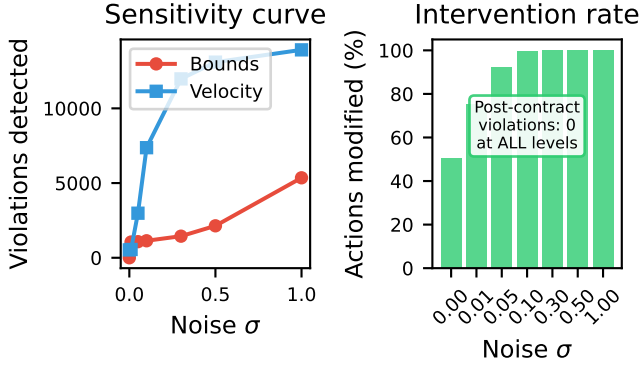


Fig. 2: Noise sensitivity calibration. Left: violations scale monotonically with noise. Right: SafeContract modifies only what is needed. Post-contract violations are zero at all noise levels.

of actions are corrected. The jerky policy (random high-frequency spikes) produces 75 velocity violations; 12% are smoothed. Post-contract output has zero OOB in all cases.

Noise sensitivity. Gaussian noise at 7 levels ($\sigma \in \{0, \dots, 1.0\}$) on 500 ground-truth LIBERO actions confirms calibration: violations scale monotonically from 527 (clean, from fast gripper snaps) to 19,269 ($\sigma=1.0$), and SafeContract eliminates 100% at every level (Fig. 2). Actions modified range from 50.6% (clean) to 100% ($\sigma=1.0$).

C. Violation Fingerprinting (EXP-H)

This experiment is the paper’s central finding: violation patterns are informative, not just corrective.

Synthetic tasks. We simulate four manipulation tasks (reaching, stacking, drawer opening, pouring), each with characteristic action distributions. Each runs for 200 steps with bounds $[-1, 1]$, $v_{\max} = 0.1$. Each task produces a unique joint-violation fingerprint (Fig. 3): reaching shows broad shoulder/elbow OOB; stacking shows wrist velocity violations; drawer shows a single-joint spike; pouring shows isolated pitch velocity violations. Fingerprints are stable across 5 seeds (CV < 0.15 for 3 of 4 tasks).

Shift detection. Concatenating two task sequences and computing violation rate in a sliding window of 20 steps, a binomial test yields $p < 0.05$ for all six task pairs ($p < 0.001$ for five of six).

Task classification. Using cosine similarity between live violation vectors and stored fingerprints, a nearest-centroid classifier achieves 100% accuracy across 40 trials.

Validation on 130 LIBERO tasks. We ran SmolVLA on all 130 LIBERO tasks: 40 from the four standard suites and 90 diverse tasks from LIBERO-90. Across the 90 diverse tasks, bounds violation rates vary from 4% (kitchen drawer manipulation) to 37% (living room mug placement). Velocity violations dominate at 72.4% of all violations, compared to 17.7% bounds violations. Rotational

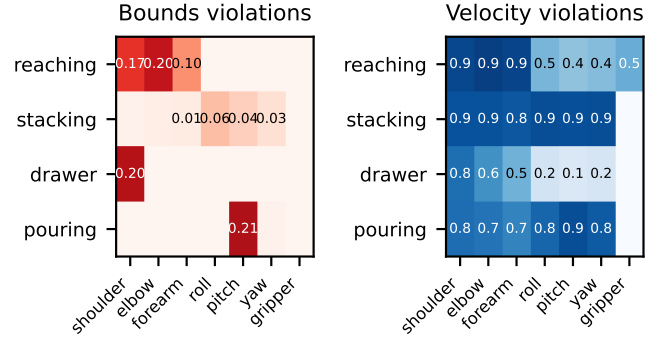


Fig. 3: Violation fingerprints across four manipulation tasks. Each task produces a distinct pattern of joint violations, enabling task identification and shift detection from safety monitor logs alone.

axes are most violated: pitch (30.6%) and roll (26.6%), while lateral motions are least affected: yaw (9.3%) and y-translation (9.7%). The standard suites show consistent patterns: spatial ($x=27\%$, $pitch=38\%$) reflects positioning demands; object ($y=18\%$, $pitch=24\%$) reflects object manipulation; goal ($pitch=38\%$, $gripper=26\%$) reflects goal-directed rotational actions; 10-task ($pitch=25\%$, $gripper=21\%$) reflects diverse multi-step coordination. These patterns emerge from the same model on genuinely different tasks, confirming that violation fingerprints reflect task-specific action distributions at scale.

V. Discussion

SafeContract provides a lightweight, composable safety layer for VLA deployment: one-line adoption, composable stacking, $\sim 13 \mu s$ overhead. The controlled ablation establishes that the monitor adds no distortion to safe policies while catching all violations in corrupted ones.

The more significant finding is that violation patterns carry diagnostic information. Standard practice treats safety enforcement as a binary gate. SafeContract’s violation logs instead create a structured behavioral profile: different tasks produce distinct fingerprints, and violation rate changes detect distribution shift with high statistical significance. This positions safety monitors as a low-cost interpretability tool requiring no gradient access or activation probing.

Normalization vs. monitoring. Action normalization (mapping model outputs to the target robot’s action space) eliminates most bounds violations. SafeContract is complementary: it provides velocity enforcement (a physical hardware constraint normalization cannot address), violation traceability for debugging, and distribution shift detection. When normalization is correct, SafeContract’s bounds checks become a passive safety net; the velocity monitoring, logging, and fingerprinting remain the primary value.

Safety-fidelity tradeoff. Tighter contracts catch more violations but clip more aggressively. On 200 synthetic sequences, violations range from 948 (very loose: ± 3.0 , $v_{\max}=1.0$, mean clip 0.03) to 2,561 (very tight: ± 0.3 , $v_{\max}=0.02$, mean clip 0.91). At the moderate operating point (± 1.0 , $v_{\max}=0.1$), SafeContract catches 2,019 violations while correcting actions by less than half a unit on average.

Behavioral telemetry for VLAs. SafeContract’s violation logs constitute a form of behavioral telemetry for VLA deployment, analogous to behavioral analytics (UEBA) in cybersecurity or statistical process control (SPC) in manufacturing. Baseline violation rates characterize normal policy behavior; deviations signal anomalies such as distribution shift, model regression, or integration errors. The 130-task LIBERO validation shows this at scale: the 4%-37% spread in violation rates across tasks, with velocity violations dominating at 72.4%, provides a deployment-time behavioral signature unavailable from training metrics. The concentration of violations on rotational axes (pitch 30.6%, roll 26.6%) versus lateral motions (yaw 9.3%, y 9.7%) reveals systematic patterns in how SmolVLA handles different kinematic dimensions. To our knowledge, no prior work treats VLA action-space constraint interactions as a telemetry signal.

Limitations. (1) EXP-G and EXP-H synthetic tasks use simulated distributions; real VLA violation patterns may differ. (2) No closed-loop evaluation: aggressive clipping could degrade task success. Preliminary closed-loop experiments yielded 0% success for both raw and contracted conditions, consistent with known checkpoint issues (LeRobot issue #2354). (3) The 100% OOB finding primarily reflects normalization mismatch. (4) The violation rate observation is empirical, not a formal statistical guarantee.

Future work. PropertyGuard: property-based testing that fuzzes action sequences against contracts. Cross-architecture fingerprint comparison (SmolVLA vs. OpenVLA vs. pi0). Conformal prediction calibration for probabilistic safety certificates. Closed-loop LIBERO evaluation with task success rates.

References

- [1] Kim, M.J. et al., “OpenVLA: An Open-Source Vision-Language-Action Model,” CoRL 2024, arXiv:2406.09246.
- [2] Shukor, M. et al., “SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics,” arXiv:2506.01844, 2025.
- [3] Cadene, R. et al., “LeRobot: State-of-the-art Machine Learning for Real-World Robotics in PyTorch,” 2024, <https://github.com/huggingface/lerobot>.
- [4] Black, K. et al., “ π_0 : A Vision-Language-Action Flow Model for General Robot Control,” arXiv:2410.24164, 2024.
- [5] Meyer, B., “Applying ‘Design by Contract,’” Computer 25(10):40–51, 1992.
- [6] Liu, B. et al., “LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning,” NeurIPS 2023.
- [7] Zhang, B. et al., “SafeVLA: Towards Safety Alignment of Vision-Language-Action Model via Constrained Learning,” NeurIPS 2025, arXiv:2503.03480.
- [8] Hu, S. et al., “VLSA: VLA Models with Plug-and-Play Safety Constraint Layer,” arXiv:2512.11891, 2025.
- [9] Kapoor, P. et al., “Constrained Decoding for Robotics Foundation Models,” arXiv:2509.01728, 2025.
- [10] Guo, J. et al., “On Robustness of Vision-Language-Action Models against Multi-Modal Perturbations,” ICLR 2026, arXiv:2510.00037.
- [11] Yang, Z. et al., “Plug in the Safety Chip: Enforcing Constraints for LLM-driven Robot Agents,” ICRA 2024, arXiv:2309.09919.
- [12] Xiao, W. et al., “SafeDiffuser: Safe Planning with Diffusion Probabilistic Models,” ICLR 2025, arXiv:2306.00148.
- [13] Ma, H. et al., “CoDiG: Constraint-Aware Diffusion Guidance for Robotics,” CoRL 2025, arXiv:2505.13131.
- [14] Tolle, M. et al., “Towards Safe Robot Foundation Models Using Inductive Biases,” arXiv:2505.10219, 2025.
- [15] Fei, S. et al., “LIBERO-Plus: In-depth Robustness Analysis of VLA Models,” arXiv:2510.13626, 2025.
- [16] Zhang, B. et al., “VLA-Arena: An Open-Source Framework for Benchmarking VLA Models,” arXiv:2512.22539, 2025.
- [17] Bhardwaj, V.P., “Agent Behavioral Contracts,” arXiv:2602.22302, 2026.
- [18] Agia, C. et al., “Unpacking Failure Modes of Generative Policies: Runtime Monitoring of Consistency and Progress,” CoRL 2024, arXiv:2410.04640.
- [19] Wu, X. et al., “SABER: A Stealthy Agentic Black-Box Attack Framework for VLAs,” arXiv:2603.24935, 2026.